

G.ai: Parent's Personalized Secretary

Team Name: G.ai

Team Members:

Ariv Gupta (UID: 805985420)

Ashita Singh (UID: 105985862)

Elliot Cao (UID: 006201926)

Jonathan Pierson (UID: 706130940)

Nyla Zia (UID: 006020803)

Radhika Kakkar (UID: 506011170)

GitHub Repository URL:

https://github.com/ny-laa/CS130_capstone_project

Context / Scope

Parents are constantly operating in a state of logistical overload. They are attempting to balance various aspects such as work, family needs, school coordination, appointments, and household responsibilities. Although there are certain tools (such as Google Calendar) designed to alleviate some of the load, these tools are often focused on the organization of information. The tools do not lessen the cognitive burden of deciding what to do, when to do it, and actually executing it. The reality is that many everyday tasks, whether it be booking appointments or tracking time sensitive emails, require a vast amount of manual effort, context switching, and follow up. This can result in decision fatigue and actions that are either missed or delayed.

The central problem then is not the lack of information or the organization of information, but a lack of execution support. Parents need a system that can actively take action on their behalf. While current attempted solutions still require users to be in the loop for every step, the ideal solution would strive to minimize friction by handling tasks end-to-end while still keeping the user in control when needed.

Our vision is to create G, an AI-powered agent that acts as an execution layer on top of a parent's existing digital ecosystem. G integrates directly with Google Calendar, Gmail, and other commonly used user tools. The primary interface would not be a new app, but familiar, low-friction channels: users interact with G by simply sending a text message or placing a phone call. Given a call or text, G would be able to take in natural language requests as input, initiate actions (such as calling to book an appointment), track progress, provide updates to the user, and loop in the user if it becomes necessary. The scope of this project includes task delegation, real-time communication, daily summaries, and controlled decision-making through human-in-the-loop escalation. The system overall aims to maximize parental productivity and minimize cognitive load.

Background INFO

The Problem with Parental Scheduling

- Parents are constantly making small decisions throughout the day: remembering to call the dentist, checking availability before booking a party, following up on a school email
- Each task is small on its own, but together they create a kind of mental drain that builds up over time
- The more decisions made throughout the day, the harder the later ones get, which is well-documented in psychology as decision fatigue
- What tools already exist:
 - Apps like Google Calendar, Cozi, and Fantastical are great at storing and displaying information, but they do not actually do anything
 - They remind parents to call someone but do not make the call. They show a conflict but do not resolve it
 - The gap is not organization, it is execution. Parents still have to context-switch between apps, make calls themselves, and remember to follow up
 - G is designed to close that gap
- Why now?
 - A few years ago, building something like G would have required a massive engineering effort
 - Today, large language models can understand natural language, break requests into steps, use external tools like calendars and phone APIs, and hold back-and-forth conversations
 - Frameworks like LangChain make it possible to wire these pieces together without building everything from scratch, which is what makes G realistic as a six-week project

Goals & Non-Goals

User Stories

User Story 1: Task Handoff

As a parent, I want to be able to text G with a task (ex. remind me at 3pm to pick up my son) so that I don't have to worry about remembering this myself.

Acceptance Criteria:

Given the parent has G installed, when the parent sends an SMS text message with task information to G, then G will extract task type (send reminder text message), content (pick up son), and time period for task (3pm).

User Story 2: Knowledge base

As a parent, I want G to have my family members' names and basic preferences so that G is able to use this context.

Acceptance Criteria:

Given the parent has downloaded G, when the parent provides basic information about his/her family (such as names) onto the setup page for G, then G will take this information and store it into a knowledge base that it will reference every time it completes a task.

User Story 3: Google Calendar

As a parent, I want G to have my access to my google calendar so that G is able to access my schedule for each day.

Acceptance Criteria:

Given the parent has downloaded G, when the parent toggles the button to provide Google Calendar access to G, then G will be able to read the parent's calendar entries.

User Story 4: Daily Updates

As a parent, I want to receive a daily morning summary of my schedule in a text message, so that I can start my day with a list of what I need to do (depends on User Story 3).

Acceptance Criteria:

Given the parent has provided G access to google calendar, when the parent toggles the button "Send Daily Updates" on the setup page, then G will read the Google Calendar information and send the parent a text message with all tasks for the current day on every morning.

User Story 5: Call Communication

As a parent, I want to be able to call G so that I do not have to type out what I want to tell G.

Acceptance Criteria:

Given the parent has downloaded G, when the parent calls G, then G will be able to use NLP to extract the parent's words.

User Story 6: Call Communication 2

As a parent, I want G to understand me when I call it to do something so that I can tell G to complete a task (depends on User Story 5).

Acceptance Criteria:

Given the parent has downloaded G, when the parent gives G a task that is supported by our product (tasks include: sending reminders, understanding texts/calls, accessing/modifying google calendar, etc.) over call and G has extracted the task words, then G will understand the task/request details.

User Story 7: Task Execution

As a parent, I want G to remind me to pick up my son from practice after I asked for it earlier (depends on User Story 1, 2).

Acceptance Criteria:

Given the parent has G installed, when the parent tasks G to send a reminder text, then G will send a reminder message at 3pm to pick up her son.

User Story 8: Pause

As a parent, I want G to pause when a task requires a decision I have not approved so that G does not complete an action I would not approve.

Acceptance Criteria:

Given the parent has given G a task, when G does not have enough information to complete it, then it will pause further execution for that task.

User Story 9: Request More Information

As a parent, I want G to ask me if it needs approval for something so that it does not execute a decision I would not approve of (depends on User Story 8).

Acceptance Criteria:

Given the parent has given G a task, when G does not know how to proceed/not have approval (ex. deleting an event from google calender to make space for something else), then it will send a text message to the parent with an approval query.

Given that the parent informs G to delete/move a calendar event, when G is about to delete/move the event, then it will send a text message to the parent confirming the new event detail and time before making changes to the calendar and ask for approval.

Given that the parent has given approval to delete/move a calendar event, when G receives the approval, then it will update the calendar to reflect the changes.

QUALITY ATTRIBUTE SCENARIOS**Quality Attribute Scenario 1: Text Response Performance**

- Stimulus: A parent sends G a task via text.
- Source: Parent user.
- Environment: Stable network.
- Response: G understands tasks given via text and responds via text.
- Response measure: Outbound text with confirmation sent within 60 seconds of the parent sending a text message.

Quality Attribute Scenario 2: Call Response Performance

- Stimulus: A parent sends G a task via call.
- Source: Parent user.
- Environment: Stable network.
- Response: G parses words from parent and understands tasks, and responds via text with confirmation.
- Response measure: Outbound confirmation text sent within 120 seconds of the phone call ending.

Quality Attribute Scenario 3: Google Calendar Security

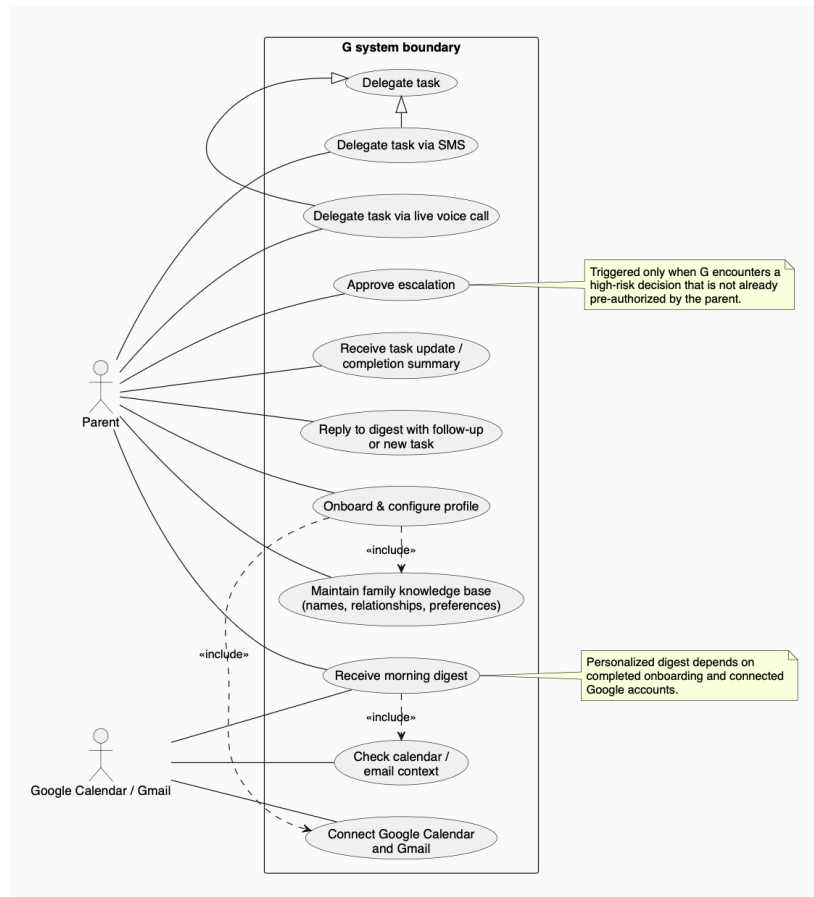
- Stimulus: A third-party attempts to login to parent's Google Calendar.
- Source: Third party.
- Environment: Normal operating conditions.
- Response: G denies the request and invalidates the token.
- Response measure: G's access token to calendar reissued every 24 hours with no interruptions.

NON-GOALS

The following are out of scope for the six-week implementation:

- Multi-user / co-parent account sharing (accounts are single-user for the initial build)
- Native iOS or Android app (web portal + SMS/Twilio is the full interface)
- Support for platforms other than Google Calendar and Gmail (no Outlook, Apple Calendar, etc.)
- Financial transactions or purchases on the user's behalf
- Multilingual support (English only)
- Learning or personalization from interaction history beyond explicit preferences

UML USECASE DIAGRAM



MOCKUPS

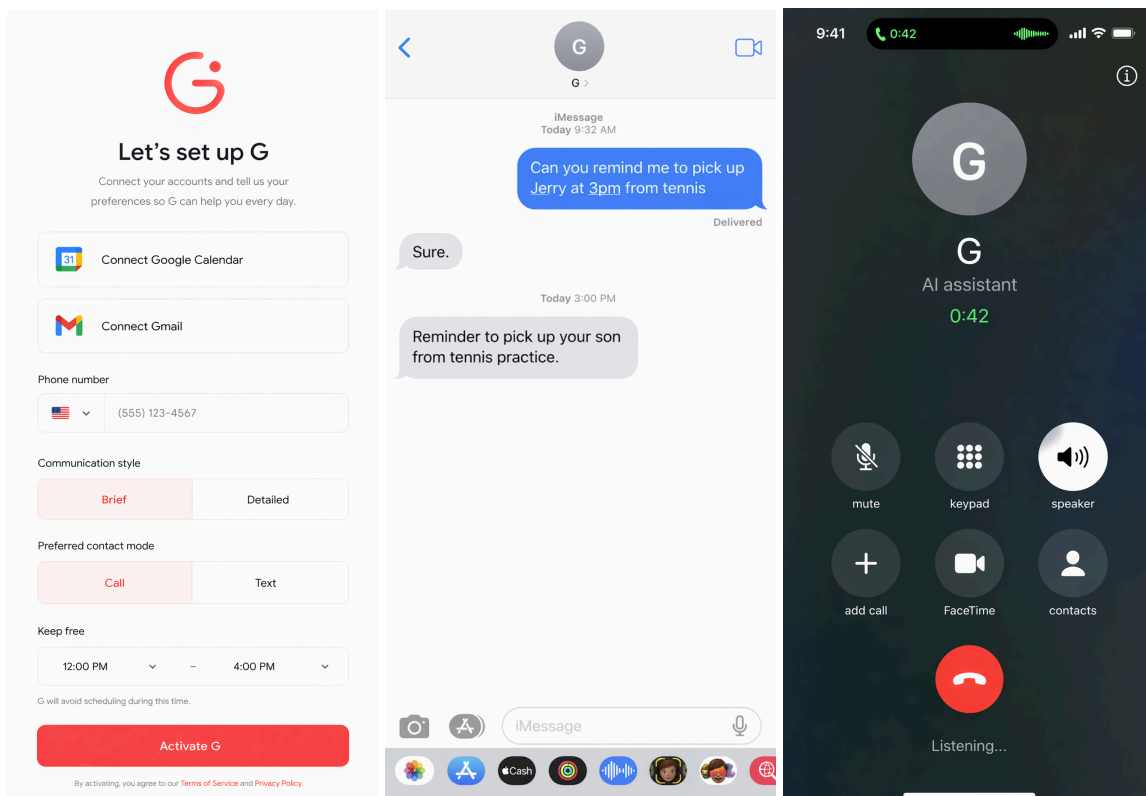


Image on the left: sign up page for setting up G. There is an option to connect Google Calendar and Gmail and preferences with how G will communicate. Image in center: Example interaction with parent requesting G to remind them to pick up their son Jerry. G reminds parent at 3:00 PM just as requested. Image on right: Example of how a voice call with G will look like with parent.

USER INTERACTION SCENARIOS:

User Interaction Scenario 1:

John has been working from 9am to 6pm for the past three days due to an emergency at his company. He is exhausted and burnt out. However, today is also his daughter's 11th birthday. He promised to buy her a huge teddy bear as a birthday present. Just as John is about to head home straight from work, G calls him. John, not knowing what the issue is, picks up the call.

G: "Good evening, John. It is Kathy's 11th birthday today. According to the notes you had, you want to be reminded to buy a big tidy bear as a present. I hope you have a wonderful evening."

Realizing that he has forgotten such an important occasion, John rushes to the mall and buys a huge teddy bear as promised for his daughter. His daughter is extremely happy about this.

User Interaction Scenario 2:

Mary needs to book an appointment with the dentist for her child. Assuming that she has already registered an account with us with her phone number, she would call our registered number for G and say what she want:

MARY: "Hey, I think my child needs to go to the dentist for a check up. Add it to my calendar so I don't forget"

G will be able to give a real time response by first analyzing her profile information (how many children she has, where she lives etc.) and her current calendar (on which day she and her child is free, etc.). After considering all the necessary information, if there are needed clarifications, Gee.ai will ask her in real time:

G: "Is Lucy or Mark the child who needs the appointment?"

MARY: "Mark."

G will assess that it has enough information.

G: "Sounds good. When is the appointment going to be? Is it with the same Doctor Lee at UCLA Hospital?"

MARY: "Next Tuesday at 3pm. Remind me an hour early. Yes it's with him."

G: "My apologies, but you currently have an event planned next Tuesday from 3pm to 4pm: Call Tim. Do you want me to cancel or move it for you?"

MARY: "Yes. Move that to 6pm. Thanks"

G: "My pleasure. I will move the event call Tim to 6pm, next Tuesday. I will send you the updated event details through SMS in a while for approval. I hope you have a wonderful day."

G will rearrange the calendar and send the updated event details and the new event details to Mary. Mary confirms, and the task is completed and a future appointment task is added to Mary's calendar.

On next Tuesday, G will text/call Mary based on her preference to remind her of the appointment.

The Design

High Level Design Description:

G is constructed around an orchestrator-agent architecture layered on top of a user's existing digital ecosystem. This system is organized into 5 layers:

- **Communication Layer (Twilio):** This is the entry and exit point for any user interaction. Twilio receives the inbound voice calls and SMS and then delivers them via webhook to the backend. It will then send outbound SMS notifications and places outbound calls on the user's behalf.
- **Orchestrator (LangChain Agent):** This component is responsible for coordination. Upon receiving a task it calls the LLM adapter to generate an execution plan and then sends each step to the appropriate tool. It is also responsible for managing task lifecycles, detecting escalation conditions and resumes paused tasks on user reply.
- **LLM Backend (Adapter Pattern):** Works to decouple the orchestrator from any specific LLM. The initial implementation targets Claude, however, the adapter makes swapping to Gemini a single file change without a need to modify the orchestrator.

- Users interact exclusively via SMS or phone calls (channels that they will already be using), while the web portal is reserved for one time setup. G is intended to operate as a background persistent agent as it accepts delegated tasks and executes them autonomously, escalating only when a decision necessitates human judgement (closing the loop with a confirmation message).

```

sequenceDiagram
    participant Parent
    participant TwilioSMS as Twilio SMS
    participant TwilioVoice as Twilio Voice
    participant SpeechProcessing as Speech Processing Service
    participant BackendWebhook as Backend Webhook API
    participant TaskOrchestrator as Task Orchestrator
    participant LLMPlanner as LLM Planner
    participant PreferenceStore as Preference Store
    participant GoogleCalendar as Google Calendar Adapter
    participant RiskEscalation as Risk / Escalation Engine
    participant TaskDatabase as Task Database

    Note over TaskOrchestrator: Incoming SMS path
    TwilioSMS->>BackendWebhook: sendSmsTo: PhoneNumber, body: String
    BackendWebhook->>TaskOrchestrator: receiveSmsWebhookFrom: PhoneNumber, body: String, messageId: String
    TaskOrchestrator->>BackendWebhook: createIncomingMessage: userId: UUID, body: String, source: SMS
    BackendWebhook->>TaskOrchestrator: handleIncomingMessage: userId: UUID, rawText: String
    TaskOrchestrator->>TaskDatabase: createIncomingCallRecord: userId: UUID, callId: String, source: VOICE

    Note over TaskOrchestrator: Incoming voice path
    TwilioVoice->>BackendWebhook: receiveVoiceWebhookFrom: PhoneNumber, callId: String
    BackendWebhook->>TaskOrchestrator: streamAudio: callId: String, audioStream: AudioStream
    TaskOrchestrator->>BackendWebhook: returnTransport: callId: String, transport: Transport
    BackendWebhook->>TaskOrchestrator: handleIncomingMessage: userId: UUID, rawText: String
    Note over TaskOrchestrator: SMS and voice converge at handleIncomingMessage().  
SMS text is used directly.  
Voice input is transcribed before orchestration begins.

    Note over TaskOrchestrator: Task planning and execution
    TaskOrchestrator->>LLMPlanner: extractIntent: rawText: String, userContext: String
    LLMPlanner-->>TaskOrchestrator: returnTaskDraft: plan: StructuredTaskPlan
    TaskOrchestrator->>PreferenceStore: getUserPreferences: userId: UUID
    PreferenceStore-->>TaskOrchestrator: returnPreferences: references: UserPreference
    TaskOrchestrator->>GoogleCalendar: checkConflicts: userId: UUID, candidateTimes: TimeWindow
    GoogleCalendar-->>TaskOrchestrator: returnAvailability: result: AvailabilityResult
    TaskOrchestrator->>RiskEscalation: generateProposedAction: plan: StructuredTaskPlan, availability: AvailabilityResult
    RiskEscalation-->>TaskOrchestrator: returnProposedAction: action: ProposedAction
    TaskOrchestrator->>TaskDatabase: classifyDecision: taskId: UUID, action: ProposedAction, prefs: UserPreference
    TaskDatabase-->>TaskOrchestrator: returnRiskDecision: decision: RiskDecision
    TaskOrchestrator->>TaskDatabase: generateSpokenResponse: action: ProposedAction, decision: RiskDecision
    TaskDatabase-->>TaskOrchestrator: returnResponseText: responseText: String
    TaskOrchestrator->>SpeechProcessing: synthesizeSpeech: callId: String, text: String
    SpeechProcessing-->>TaskOrchestrator: streamAudio: response: callId: String, audioStream: AudioStream
    TaskOrchestrator->>TwilioVoice: playAudio: audioStream: AudioStream

    Note over TaskOrchestrator: Escalation to parent
    TaskOrchestrator->>TwilioSMS: sendEscalationRequest: userId: UUID, options: Option
    TwilioSMS->>BackendWebhook: deliverSmsTo: PhoneNumber, body: String
    TwilioSMS->>BackendWebhook: sendSmsTo: PhoneNumber, body: String
    BackendWebhook->>TaskOrchestrator: receiveSmsWebhookFrom: PhoneNumber, body: String, messageId: String
    TaskOrchestrator->>TaskDatabase: resumeTaskFromReply: userId: UUID, replyText: String
    TaskDatabase-->>TaskOrchestrator: findPending: escalation: userId: UUID, replyText: String
    TaskOrchestrator->>TaskDatabase: returnEscalation: escalation: EscalationRequest

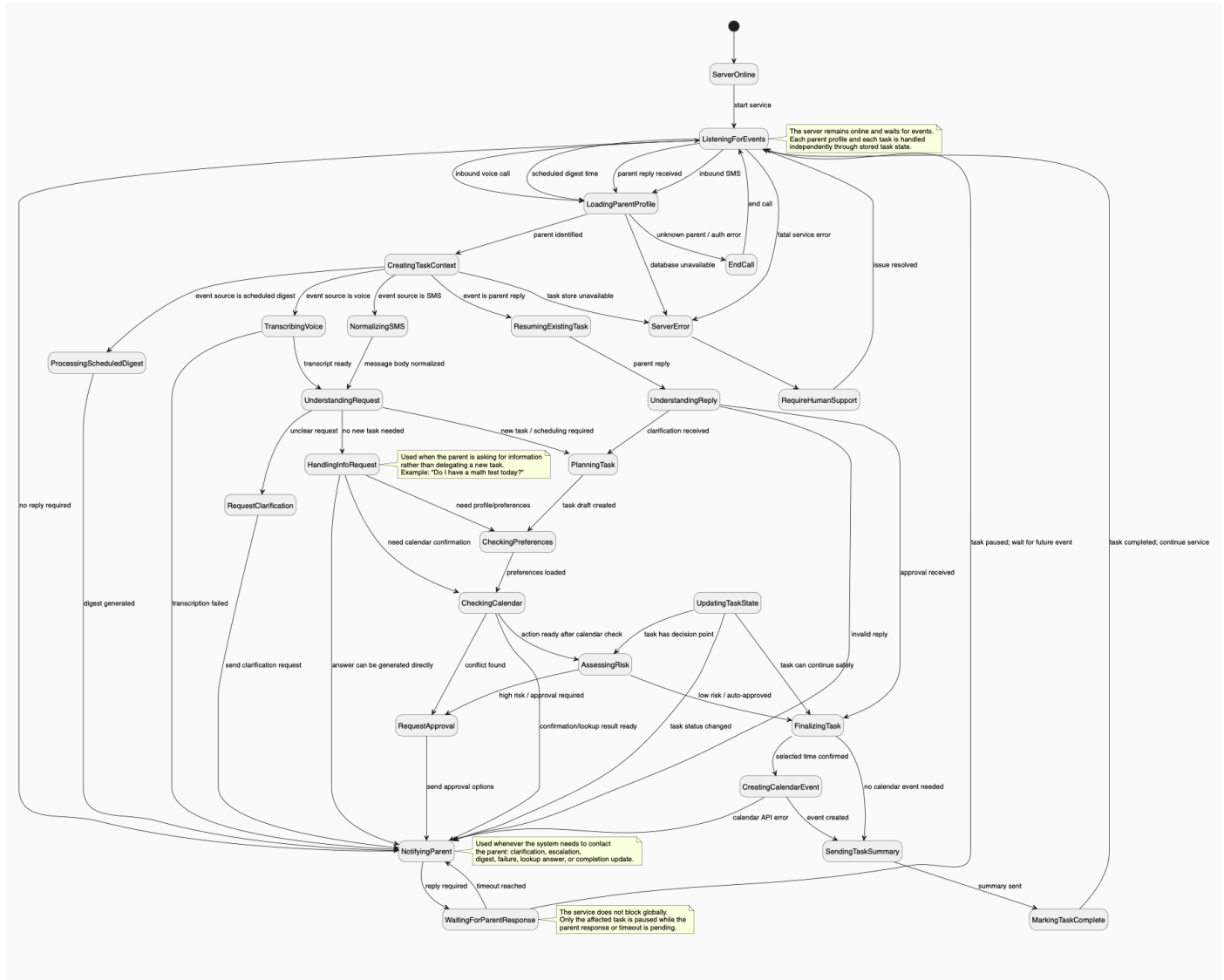
    Note over TaskOrchestrator: Completion
    TaskOrchestrator->>TaskDatabase: createEvent: userId: UUID, title: String, start: DateTime, end: DateTime
    TaskDatabase-->>TaskOrchestrator: returnEvent: eventId: String
    TaskOrchestrator->>TaskDatabase: markTaskCompleted: taskId: UUID, eventId: String
    TaskDatabase-->>TaskOrchestrator: sendCompletionSummary: userId: UUID, taskId: UUID
    TaskOrchestrator->>TwilioSMS: deliverSmsTo: PhoneNumber, body: String
  
```

Simplified version with less case by case branching:
We simplified our complete state machine graph for easier interpretation.



Complex version with more case by case branching:

Here is a slightly more complicated one that considers more scenarios and branching. You don't need to understand this, but it's very straightforward if you think about a natural process of handling everyday tasks. *This also explains the need for an AI personal secretary for parents to uplift the cognitive load everyday.*



Relevant API Descriptions

The following describes the semantics, parameters, pre/post-conditions, and error cases for G's primary interfaces and the external APIs it depends on. We may or may not alter it according to future needs during development.

POST /webhook/sms (Twilio -> Backend, SMS)

Field	Description
Purpose	Receive an inbound SMS from the parent.
Parameters	From: parent phone number in E.164 format. Body: message text. MessageSid: Twilio message id.
Preconditions	Phone number must be registered. Twilio HMAC-SHA1 signature must be valid.
Postconditions	Task record created and the orchestrator is triggered asynchronously.
Errors	Invalid signature -> reject request. Unknown phone number -> send fallback/auth message if possible. Internal error -> log and allow Twilio retry.

POST /webhook/call (Twilio -> Backend, voice)

Field	Description
Purpose	Handles an inbound phone call from a parent. The call audio is streamed to speech processing so G can respond in the call.
Parameters	CallSid: Twilio call id. From: caller number. CallStatus: current call state.
Preconditions	The caller's phone number is registered. The voice streaming service is reachable.
Postconditions	Live bidirectional audio stream active, G will transcribe and respond
Errors	If voice streaming or transcription fails, G plays a short fallback message and follows up by SMS.

GOrchestrator.delegateTask(message, userId)

This could be sent to any of the tools we have (e.g. calendar, LLM...)

Field	Description
Purpose	Parse the parent's message, generate an LLM execution plan and persist a new Task in state PENDING.
Parameters	message: str (raw text). userId: str (user UUID).
Returns	Task and task id, status=PENDING, plan_steps list.
Preconditions	userId exists in UserProfileStore and the LLM adapter is reachable.
Postconditions	Task record created and runTaskPlan() enqueued asynchronously.
Errors	LLMUnavailableError if adapter times out (>10 s). AmbiguousTaskError if intent is unclear -> G will reply asking for clarification.

CalendarTool.checkAvailability(userId, window)

Field	Description
Purpose	Query Google Calendar to determine if time window is free.
Parameters	userId: str. window: TimeWindow {start: datetime, end: datetime, timezone: str}.
Returns	Bool result: True if the window is free and False if any event overlaps.
Preconditions	Valid Google OAuth token with calendar.readonly scope on file for userId.
Postconditions	No side effects (read-only call).
Errors	TokenExpiredError triggers OAuth refresh. QuotaExceededError retries with exponential backoff (max of 3 attempts).
Units	Start and end times are stored in UTC and shown to the parent in their local timezone.

GOrchestrator.escalate(task, options)

Field	Description
Purpose	Pause task execution and send the parent a structured SMS decision prompt. Will wait for reply before resuming. Useful if G want to delete some event from calendar.
Parameters	task: Task. options: list[str]. 2-3 human-readable choices.
Preconditions	Task must be IN_PROGRESS and options must have 2-3 items.
Postconditions	Task status set to ESCALATION_PENDING. The escalation_deadline = now + 30 min. SMS delivered via SMSTool.
Errors	SMSDeliveryError: task status set to ESCALATION_FAILED; parent notified on next available channel.

Data Model

G uses a PostgreSQL database to store parent profiles, family context, preferences, task state, and communication history. We chose a relational database because the main records have clear relationships: one parent can have many family members, many tasks, and many messages. All timestamps are stored in UTC so that reminders and calendar events can be handled consistently across time zones.

users (one row per registered parent account)

Column	Type	Description
id	UUID_PK	Auto-generated primary key
phone_number	VARCHAR(20)	Parent's phone number in E.164 format, used to match incoming Twilio texts and calls
email	VARCHAR(255)	Google account email; used for OAuth identity
comm_style	ENUM	Parent's preferred response style, such as brief or detailed
preferred_channel	ENUM	Parent's preferred contact method, such as sms or call
blocked_windows	JSONB	Array of {start_time, end_time} recurring blocks
calendar_token	TEXT	Encrypted Google OAuth token (Calendar scope)
gmail_token	TEXT	Encrypted Google OAuth token (Gmail scope)
created_at	TIMESTAMPTZ	Account creation timestamp (UTC)

tasks (one row per delegated action)

Column	Type	Description
id	UUID_PK	Auto-generated primary key
user_id	UUID_FK	References users.id (the user for which this task is for)
status	ENUM	PENDING IN_PROGRESS ESCALATION_PENDING COMPLETED FAILED
type	VARCHAR(50)	General task category, such as reminder, calendar_update, information_request, or morning_digest
description	TEXT	Original natural-language request
plan_steps	JSONB	Ordered array of {tool, params, status} objects
escalation_deadline	TIMESTAMPTZ	NULL unless ESCALATION_PENDING; 30-min timeout
created_at	TIMESTAMPTZ	UTC task creation time
updated_at	TIMESTAMPTZ	UTC timestamp of last state change

messages (audit log of all communications)

Column	Type	Description
id	UUID_PK	Auto-generated primary key
task_id	UUID_FK	References tasks.id (nullable for unsolicited messages)
direction	ENUM	'inbound' or 'outbound'
channel	ENUM	'sms' or 'voice'
content	TEXT	Raw message body or voice transcript
timestamp	TIMESTAMPTZ	UTC send/receive time

preferences (key-value per-user settings, one user can have multiple key-value pairs on the same row.)

Column	Type	Description
user_id	UUID_FK	References users.id
key	VARCHAR	e.g. 'digest_time', 'auto_confirm_categories'
value	TEXT	Serialized preference value

Tech Stack

We chose a stack that lets us build the prototype quickly without writing every integration from scratch. Since the project depends heavily on SMS, voice, calendar access, and stored task state, most of our choices are existing services with stable APIs. To keep the scope realistic, we are avoiding a native mobile app and focusing on a web onboarding portal plus phone/SMS interaction.

Layer	Technology	Rationale
Web framework	FastAPI	Async-native, handles concurrent Twilio webhooks cleanly.
Orchestration	LangChain	Agent loop, tool dispatch and memory abstractions.
LLM (primary)	Claude API (Anthropic)	Strong tool-use and instruction-following. Accessed using ClaudeAdapter.
LLM (fallback)	GPT-4 (OpenAI)	Available via GPTAdapter and the swap requires no orchestrator changes.
SMS / Voice	Twilio	Industry standard and already tested basic SMS webhook locally.
Speech-to-text	Deepgram	Real-time streaming
Calendar / Email	Google Calendar + Gmail APIs	Native integrations for the parent's primary calendar and email.
Database	PostgreSQL (Supabase)	Relational model fits task/user structure
Task queue	Celery + Redis	Decouples webhook response from async task execution.
Frontend (portal)	React + Vite	Lightweight SPA for onboarding
Hosting	Google Cloud Run	Serverless containers and scales to zero between tasks, minimizing cost.
Auth	Google OAuth 2.0	Single sign-on, grants Calendar and Gmail scopes in one flow.

Alternatives Considered

Alternative 1:

An alternative design considered was the creation of a dedicated mobile app interface. Users would be able to interact with G through a structured dashboard, opposed to text messaging or calls. This would be a more tailored and optimized version of similar tools such as Notion. The dashboard would include a centralized view of tasks, timelines, and agent activity, offering greater transparency and easier state tracking. However, after further investigation, we realized that creating a new app would introduce adoption friction since users would be required to install and navigate a new tool, which contradicts the overall goals of minimizing effort and context switching for busy parents. In contrast, our final design's SMS/voice-first approach allows users to interact with G through familiar, low-friction channels, making task delegation faster and more seamless within their existing daily workflows.

Alternative 2:

Another alternative design considered was to build a custom agent loop directly against the Claude API instead of using LangChain. This would provide complete control over the agent's behavior and avoids the abstraction overhead and versioning instability that can come with using LangChain. However, through LangChain's AgentExecutor, tool abstractions and retry logic, it would drastically decrease the amount of time needed to get a working task delegation loop which is highly important given the six week time constraint. LangChain also provides conversation memory and step tracking which would be another time consuming task to be completed. Lastly, familiarity played a key role in the decision as several team members had worked with LangChain previously which would make getting the project up and running a much smoother process.

Feasibility

Familiarity with APIs

- The four main technologies G relies on are Twilio, Google Calendar API, Gmail API, and LangChain, all of which are well-documented with active communities
- Several team members have used LangChain and Google OAuth in previous projects
- A basic Twilio SMS webhook has already been tested locally to confirm the core flow works
- There is no API here that none of us have touched before, which meaningfully reduces the risk of getting stuck

Performance

- Most of what G does, such as sending a morning digest, booking an appointment, or checking calendar conflicts, does not have tight latency requirements
- The one exception is the real-time voice call (US-5), where a slow response feels broken
- The plan is to use Deepgram for speech-to-text since it is fast enough to transcribe while the speaker is still talking
- LLM prompts will be kept short and focused to keep response times down
- The internal target is under 3 seconds from when the speaker stops talking to when G starts responding, which is achievable with this stack

Cost

- Twilio charges per message and per minute of call time, but at prototype usage levels the cost is negligible
- Google Calendar and Gmail APIs are free within standard quota limits
- LLM API calls are the only other real cost, and with concise prompts and low call volume the project is expected to stay within free-tier or low-cost usage throughout the quarter

Novelty

- Most AI assistant products are either general-purpose (Siri, Google Assistant) or targeted at professionals and students
- The specific combination of SMS/voice task delegation, outbound calling on a parent's behalf, and human-in-the-loop escalation built around parental workflows does not exist in any product found during research
- The closest alternatives are enterprise tools that cost hundreds of dollars a month and are far too complex for personal use

Scope

- One of the most common reasons software projects fail is trying to build too much
- Multi-user support, non-Google integrations, financial transactions, and a native mobile app have all been explicitly scoped out
- The core loop, where a parent sends a text or calls and G handles the task and reports back, is narrow enough to build and test in six weeks while still being genuinely useful

Third-party Reliability

- Twilio and Google are both extremely reliable services with strong uptime records
- The one real risk is outbound calling. If G calls a business and the line is busy or hits a confusing phone tree, the system needs to fail gracefully
- This has been accounted for in US-1's acceptance criteria: G texts the parent explaining what went wrong and suggests next steps, rather than silently failing